

Hadoop

Prepared By : Asmaa Ahmed Maryah

Supervised By: Dr. Aya Gamal

Table of Contents

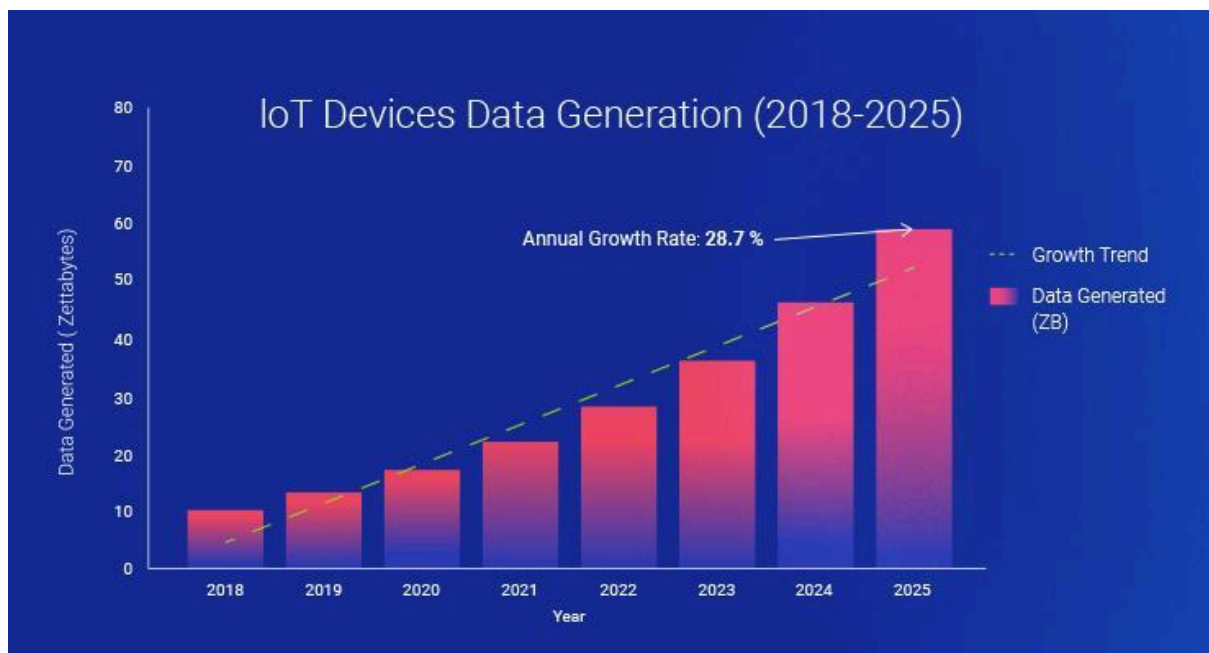
1. Introduction to Hadoop	3
2. History of Hadoop	4
3. Hadoop Ecosystem Overview	7
4. HDFS (Hadoop Distributed File System)	7
5. MapReduce	9
6. YARN	11
7. Advantages of Hadoop	14
8. Disadvantages of Hadoop	14
9. Real-world Use Cases	15
10. References	16

1. Introduction to Hadoop

Data!

We live in the data age. It's not easy to measure the total volume of data stored electronically, but an IDC estimate put the size of the “digital universe” at 4.4 zettabytes in 2013 and is forecasting a tenfold growth by 2020 to 44 zettabytes.¹ A zettabyte is 10^{21} bytes, or equivalently one thousand exabytes, one million petabytes, or one billion terabytes. That's more than one disk drive for every person in the world. This flood of data is coming from many sources. Consider the following:

- The New York Stock Exchange generates about 4–5 terabytes of data per day.
- Facebook hosts more than 240 billion photos, growing at 7 petabytes per month.
- Ancestry.com, the genealogy site, stores around 10 petabytes of data.
- The Internet Archive stores around 18.5 petabytes of data.



The Beginning of the Big Data Problem (Early 2000s)

At the start of the 2000s, the internet was growing at an extremely fast rate. Companies like **Google** were facing a major technical challenge:

how to store and index an enormous number of web pages while keeping search results fast and efficient.

Google's main goal was:

- To **index the largest possible portion of the web**
- To provide **search results in the shortest possible time (low latency / low seek time)**
- To handle continuously growing data from web pages, logs, and user activity

However, the traditional computing methods were no longer enough.

Limitations of Traditional Approaches

Before Hadoop and distributed systems, data processing relied on **single-machine or centralized systems**. These approaches had several limitations:

1. Single Machine Processing

- All data is stored and processed on one powerful server
- Limited by CPU, RAM, and storage capacity
- If the machine fails, the entire system fails

2. Vertical Scaling (Scaling Up)

- Improving performance by upgrading hardware (more RAM, faster CPU, larger disk)
- Very expensive and still limited in capacity

3. Traditional Databases (RDBMS)

- Designed for structured data only
- Not suitable for huge unstructured or semi-structured data (like web pages, logs, images)
- Slow when handling billions of records

4. Centralized Storage Systems

- Data stored in one location
- High risk of bottlenecks and failures
- Poor performance when data size grows massively

Table 1-1. RDBMS compared to MapReduce

	Traditional RDBMS	MapReduce
Data size	Gigabytes	Petabytes
Access	Interactive and batch	Batch
Updates	Read and write many times	Write once, read many times
Transactions	ACID	None

Google Research Papers (2003–2004) and Their Role in Hadoop

In 2003 and 2004, Google published two groundbreaking research papers that significantly influenced the development of distributed systems and laid the foundation for **Apache Hadoop**.

The first paper introduced the **Google File System (GFS)**, a distributed file storage system designed to handle massive amounts of data across clusters of commodity hardware. Instead of relying on a single machine, GFS splits large files into blocks and distributes them across multiple nodes. It also ensures **fault tolerance**, meaning the system can continue operating even if some machines fail. This approach allowed Google to efficiently store and manage web-scale data.

The second paper presented the **MapReduce programming model**, which simplified large-scale data processing. MapReduce divides processing into two main phases: the **Map phase**, where data is processed and transformed in parallel across multiple nodes, and the **Reduce phase**, where the intermediate results are aggregated to produce the final output. This model made it possible to process huge datasets in a scalable and efficient way.

These two innovations inspired **Doug Cutting** and his team to develop **Hadoop** as an open-source implementation of the same concepts. Hadoop adopted the ideas of distributed storage (similar to GFS through HDFS) and distributed processing (similar to MapReduce), making big data processing accessible outside of Google.

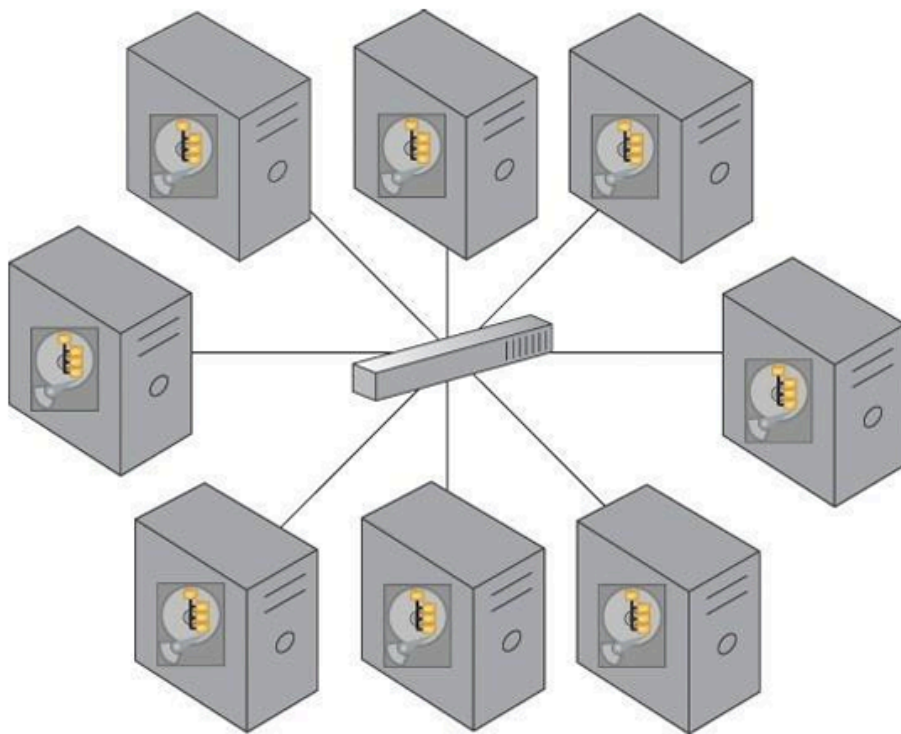
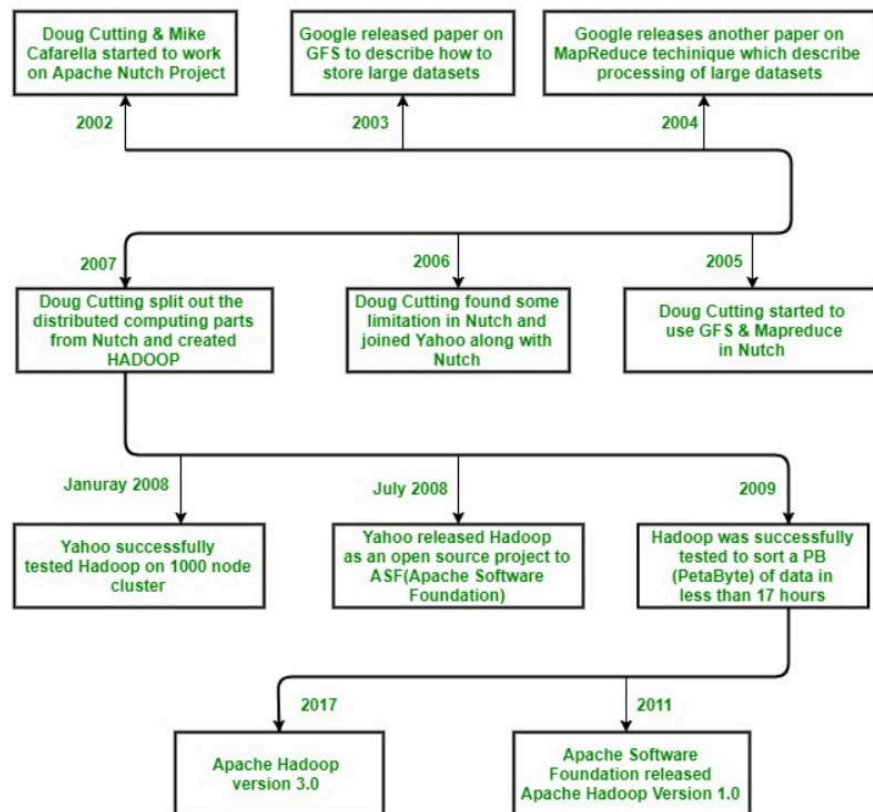


Figure 5.3 The symbol used to represent distributed file systems.

Why is it called “Hadoop”?

The name **Hadoop** comes from the toy elephant of **Doug Cutting’s son**, who was one of the original creators of the project. The elephant was named “*Hadoop*”, and Doug Cutting liked the name because it was unique, easy to remember, and had no technical meaning at the beginning.

Let's Summarize above History :



Hadoop Ecosystem Overview

The **Hadoop Ecosystem** is a collection of tools and frameworks that work together to store, process, and analyze **Big Data** in a distributed way. Instead of relying on a single system, Hadoop provides multiple components, each designed for a specific task, all integrated to handle large-scale data efficiently.

Core Idea

Instead of having a single program that performs all tasks, Hadoop divides the work into separate components:

- Data storage
- Data processing
- Resource management
- Data analysis
- Data transfer

1. Hadoop and Distributed File Systems

As data continues to grow rapidly in both size and speed, it becomes too large for a single machine to store or process efficiently. Traditional systems quickly reach their limits when handling such massive amounts of data.

To solve this problem, the idea of **Distributed File Systems (DFS)** was introduced. **Instead of storing all data in one place, the data is split and stored across multiple machines connected in a network.** This approach improves storage capacity and processing power, but it also introduces challenges such as network complexity and system reliability.

- **Extremely large files:** Here, we are talking about the data in a range of petabytes (1000 TB).
- **Streaming Data Access Pattern:** HDFS is designed on principle of *write-once and read-many-times*. Once data is written large portions of dataset can be processed any number times.
- **Commodity hardware:** Hardware that is inexpensive and easily available in the market. This is one of the features that especially distinguishes HDFS from other file systems.

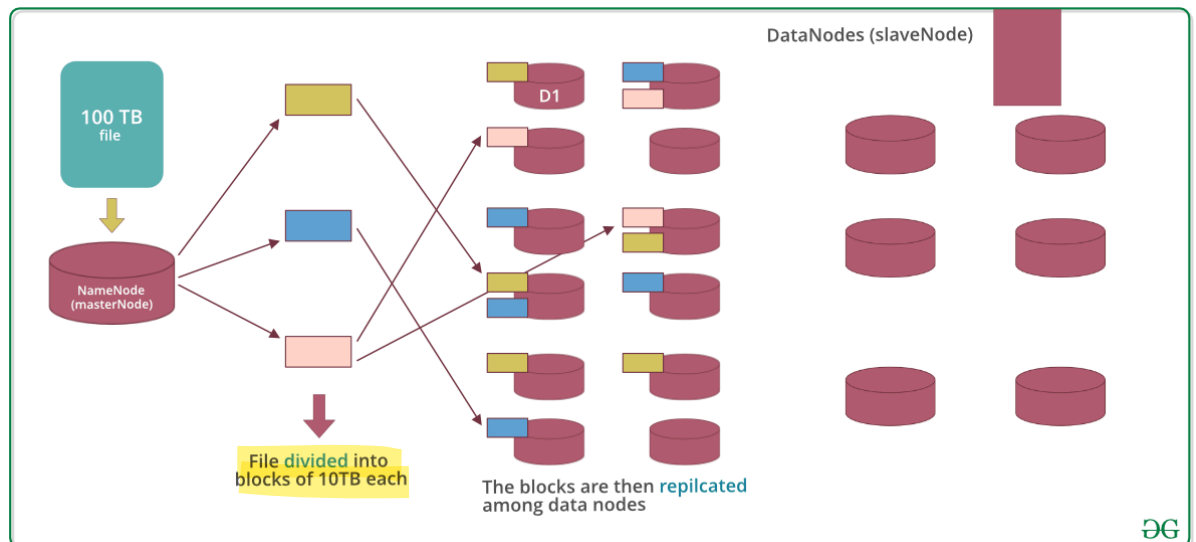
Nodes: Master-slave nodes typically form the **HDFS cluster**.

1. NameNode(MasterNode):

- **Manages all the slave nodes and assigns work to them.**
- It executes filesystem namespace operations like opening, closing, and renaming files and directories.
- It should be deployed on reliable hardware that has a high configuration. not on commodity hardware.

2. DataNode(SlaveNode):

- **Actual worker nodes do the actual work like reading, writing, processing, etc.**
- They also perform creation, deletion, and replication upon instruction from the master.
- They can be deployed on commodity hardware.



2. MapReduce Architecture

MapReduce Architecture is the backbone of Hadoop's processing, offering a framework that splits jobs into smaller tasks, executes them in parallel across a cluster, and merges results. Its design ensures parallelism, data locality, fault tolerance, and scalability, making it ideal for applications like log analysis, indexing, machine learning, and recommendation systems.

Core Components of MapReduce Architecture

1. Client

The Client is the entry point into MapReduce. It submits the job for execution by packaging the Mapper and Reducer logic into a JAR file and specifying the input and output paths. After submission, the Client's role ends.

2. Job

A Job represents the complete processing request from the client.

- Internally, it is divided into **job parts (tasks)**.
- Each job part is assigned to either the Map or Reduce phase

3. Hadoop MapReduce Master

The Master Node coordinates job execution. It:

- Accepts jobs from the Client.
- Splits them into job parts.
- Assigns these parts to the Map and Reduce tasks.
- Tracks execution progress and reassigns failed tasks.

4. Job Parts (Tasks)

Every job is divided into smaller units called job parts.

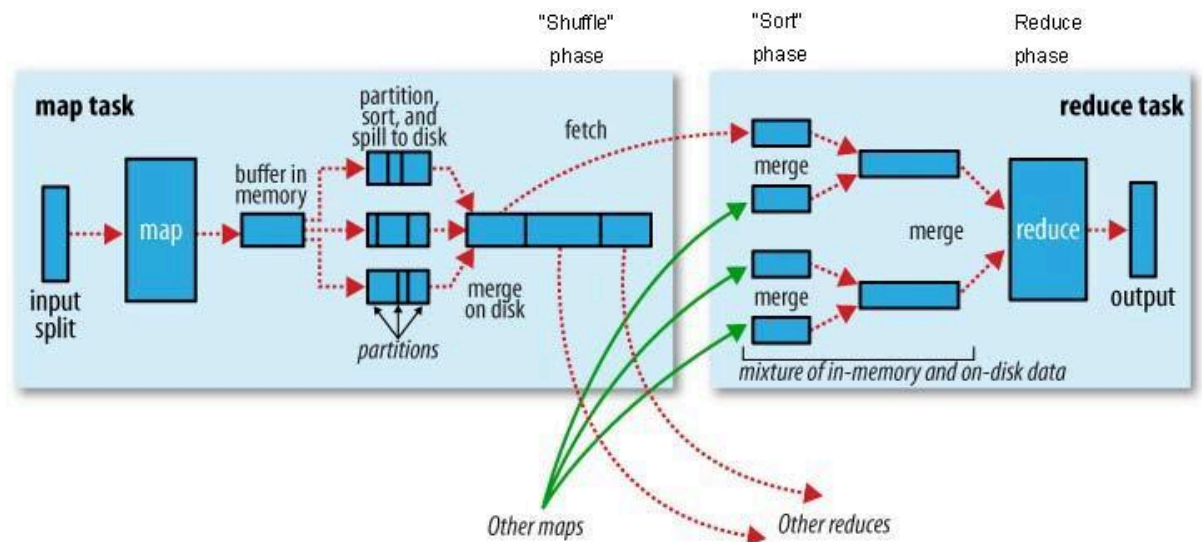
- Map job parts: Process input data splits into intermediate key–value pairs.
- Reduce job parts: Aggregate intermediate results into final outputs.

5. Map Phase

- Input data is split and given to Map tasks.
- Each Map task processes its local split and produces intermediate results in the form of key–value pairs.

6. Shuffle & Sort (Between Map and Reduce)

- Intermediate key–value pairs from the Map phase are grouped by key.
- Sorting ensures ordered keys before passing them to the Reducers.

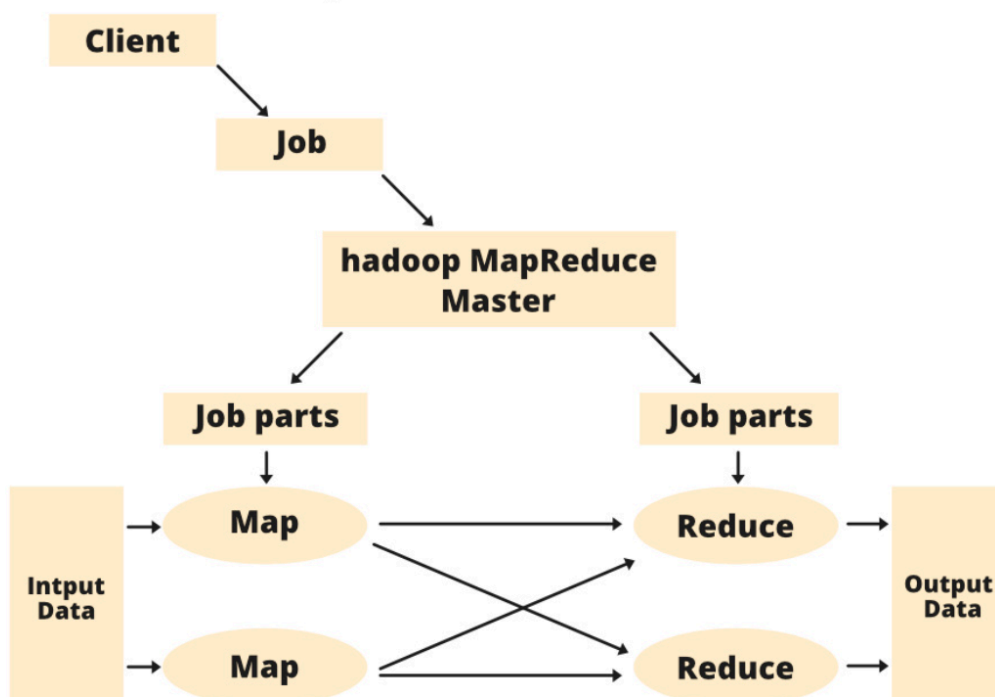


7. Reduce Phase

Reducers take grouped keys and values from Shuffle & Sort.

They aggregate them to produce the final output data, which is written back to HDFS.

Map Reduce Architecture



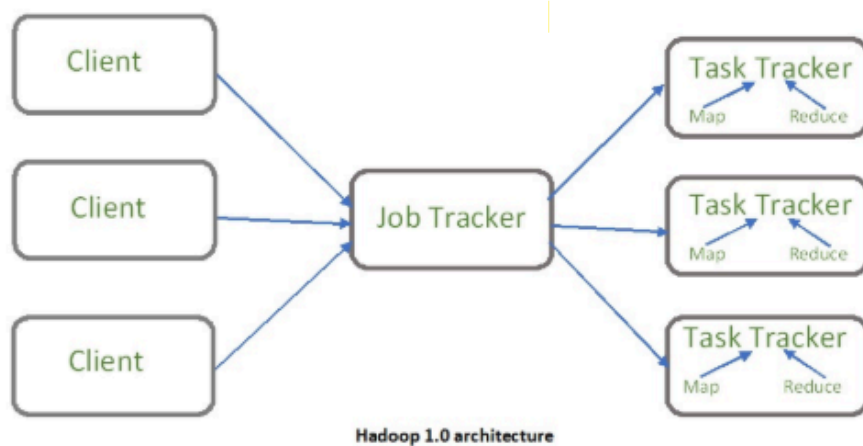
3. Hadoop YARN Architecture

YARN (Yet Another Resource Negotiator) is a core component of Hadoop introduced in **Hadoop 2.0**. Its main purpose is to manage computing resources and improve the scalability of Hadoop compared to the older architecture.

Why YARN was Introduced?

In Hadoop 1.0, there was a major limitation:

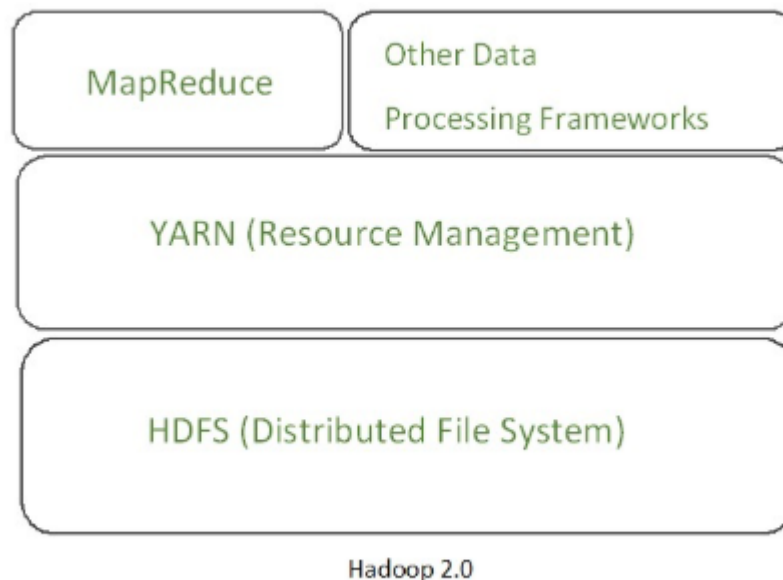
- The **JobTracker** handled both:
 - Resource management
 - Job scheduling and monitoring



What is YARN?

It acts as a **resource management layer** in Hadoop that allows multiple applications to run and share cluster resources efficiently.

1. **Manages resources**
2. **Schedules tasks**
3. Supports multiple processing frameworks (not just MapReduce)



Hadoop YARN Architecture (Explanation of the Diagram)

Clients

Clients represent users or applications that submit processing jobs to the Hadoop system. Multiple clients can send requests at the same time.

Resource Manager

The **Resource Manager** is the main component of YARN. It acts as the central coordinator of the cluster and is responsible for:

- **Receiving job requests from clients**
- Managing and allocating system resources
- Deciding where each job should run in the cluster

Node Managers

Each **Node Manager** runs on a worker machine in the cluster.
It is responsible for:

- Executing tasks assigned by the Resource Manager
- Managing system resources on that node
- Running containers that process data

Application Master (App Master)

Each job has its own **Application Master**, which is launched inside a Node Manager.

Its responsibilities include:

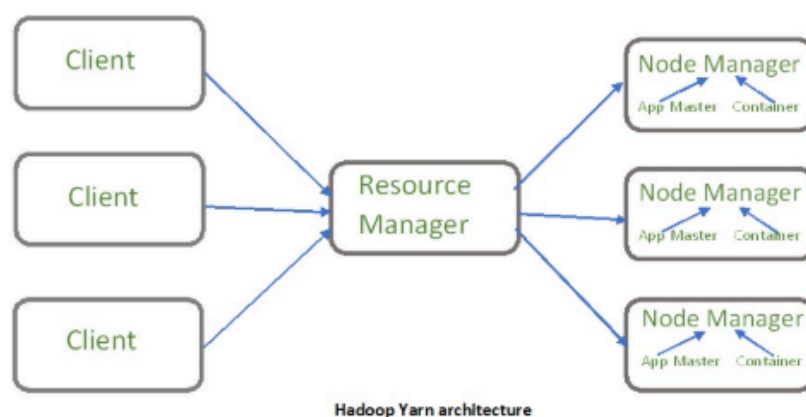
- Managing a single application (job)
- Coordinating task execution for that job
- Communicating with the Resource Manager for resources

Containers

Containers are the actual execution environments where tasks run.

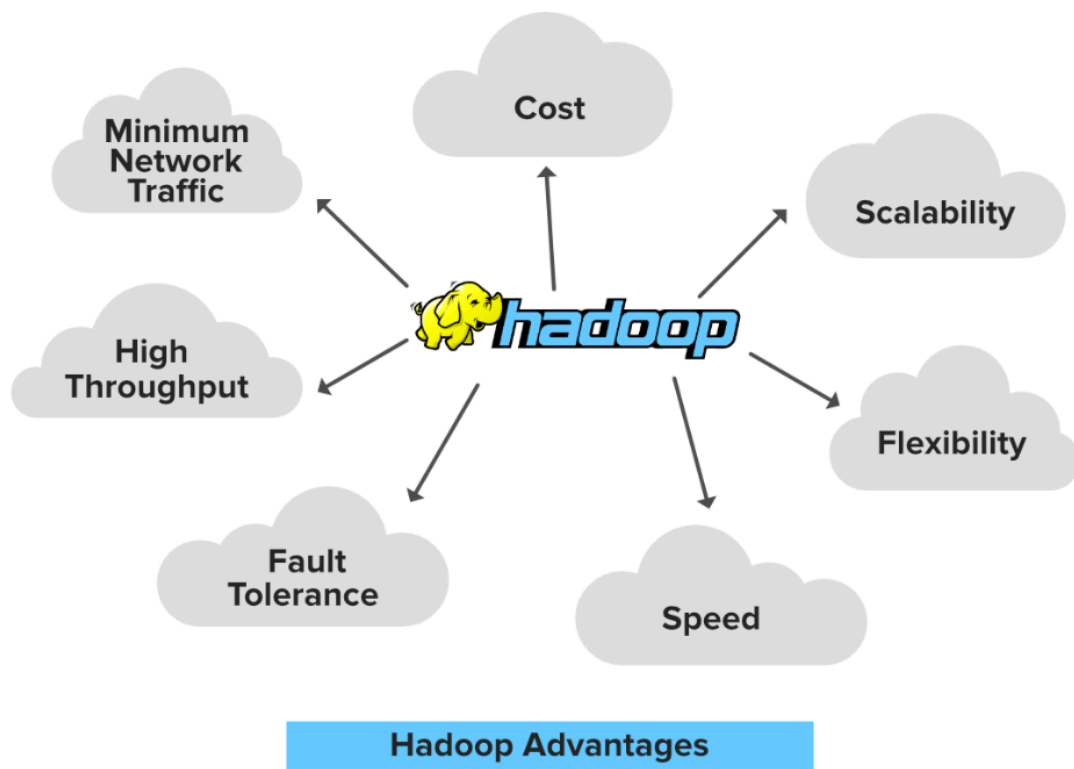
Each container provides:

- CPU resources
- Memory resources
- Execution space for tasks

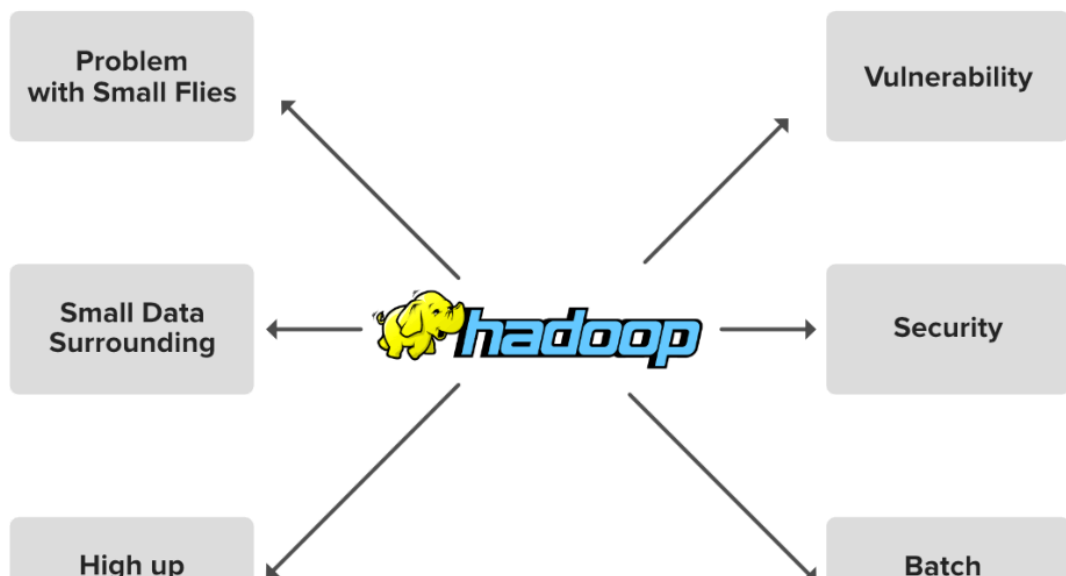


Hadoop - Pros and Cons

Advantages of Hadoop



Disadvantages of Hadoop



Real-World Use Cases of Apache Hadoop

1. E-commerce Industry

Hadoop is used to analyze large volumes of customer interaction and transaction data.

Applications include:

- Customer behavior analytics and segmentation
- Recommendation systems (product suggestion engines)
- Sales trend analysis and demand forecasting
- Clickstream analysis (user navigation tracking)

2. Social Media Platforms

Social networks generate massive unstructured data (posts, images, videos, logs).

Applications include:

- Sentiment analysis of user-generated content
- Trend detection and viral content identification
- Social graph analysis
- Spam and fake account detection

4. Media and Entertainment

Streaming platforms process massive user interaction and content data.

Applications include:

- Recommendation systems for movies and videos
- User behavior and viewing pattern analysis
- Content performance tracking
- Personalized content delivery

References

1. Hadoop The Definitive Guide Book
2. Geeks for Geeks website
3. [Big-Data بالعربي](#)